

AUTOMATED JOURNALIST APP

Technical University of Munich

Ecem Varma, Ahmet Bilal Akın, Can Bilgin

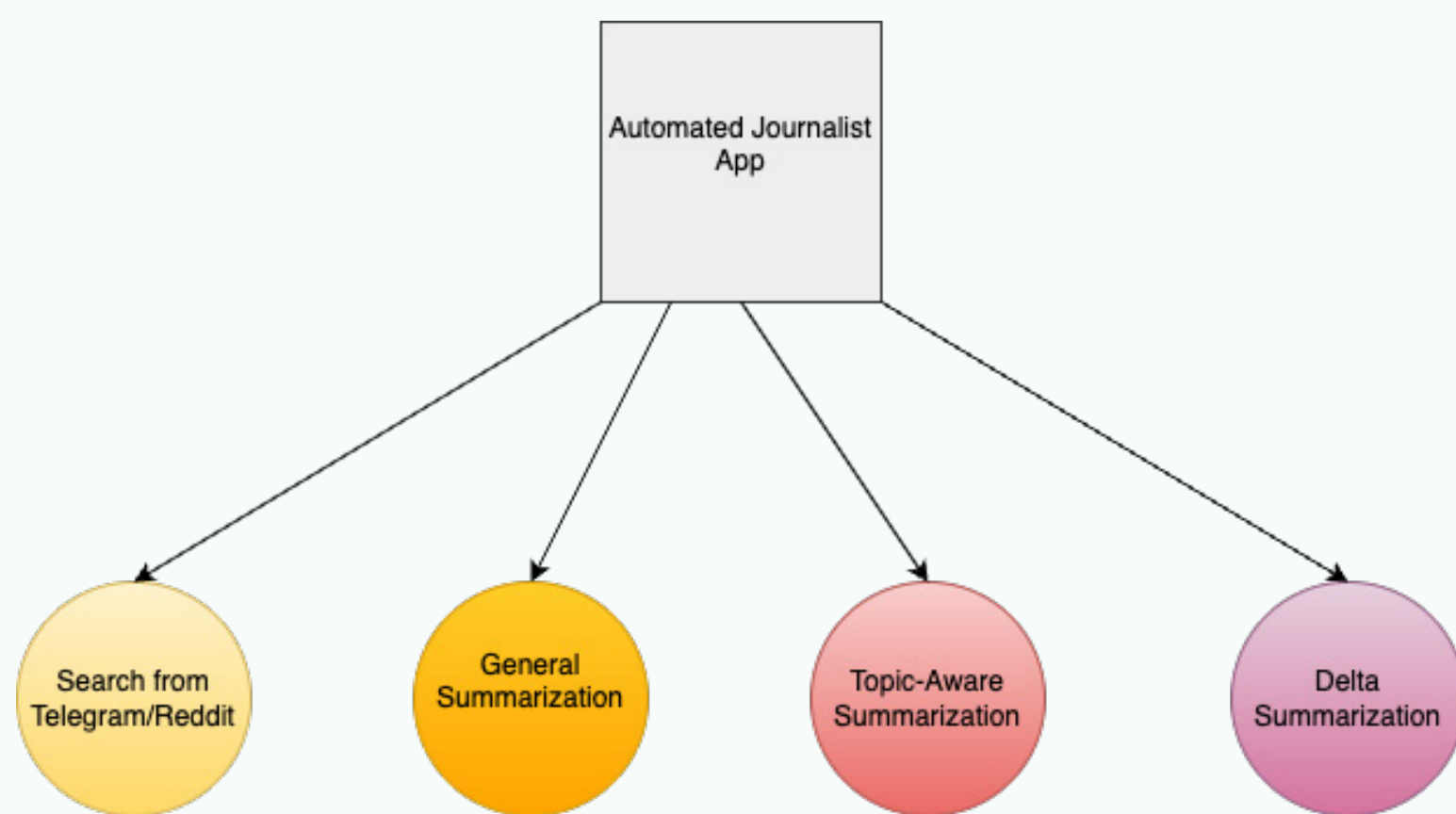


INTRODUCTION

In the age of information overload, journalists face the challenge of staying informed with accurate and relevant updates. Social media platforms, while offering a wealth of real-time information, often present data in a scattered and overwhelming manner. To address this, the Automated Journalist App has been designed to collect and summarize social media content using advanced Natural Language Processing (NLP) techniques.

The app offers powerful features, including the ability to search content from Telegram and Reddit, perform general summarization, and provide topic-aware summarization and delta summarization, which involves comparison with respect to topics. Leveraging BERT (Bidirectional Encoder Representations from Transformers), it delivers concise and coherent summaries.

This app bridges the gap between the vast information available on social media and the need for digestible, actionable insights. With the Automated Journalist App, journalists can access timely, accurate, and contextually relevant information. Whether for conducting research, writing articles, or staying updated on trends, this app showcases the potential of AI-driven solutions in transforming how media professionals consume and interpret social media data.



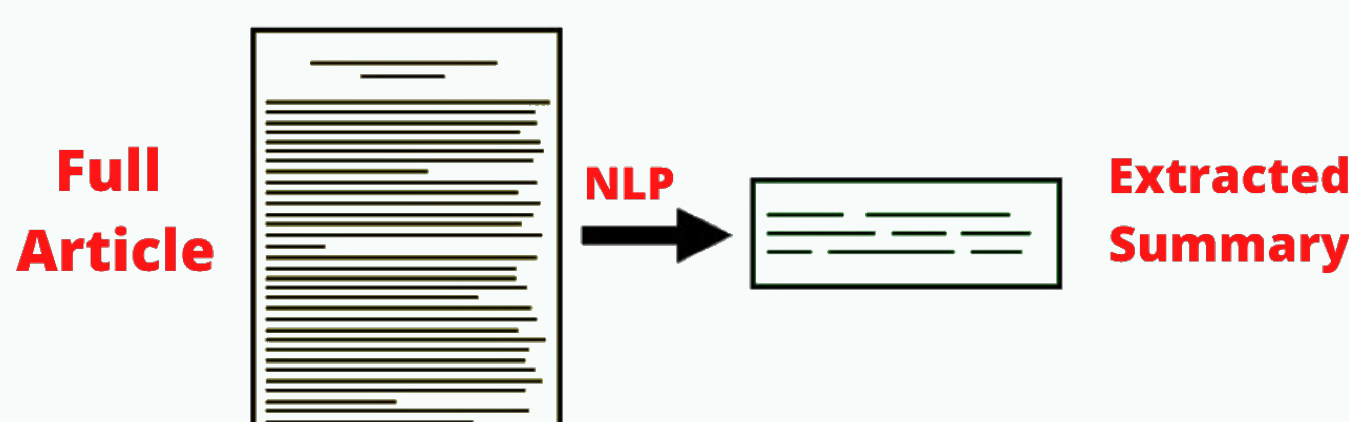
METHODS

TEXT SUMMARIZATION

In our Automated Journalist App, our primary task is to summarize text efficiently. We leverage the state-of-the-art BART model for this purpose. The process involves two main steps: preprocessing and chunking the input conversations.

- Preprocessing:** We clean and prepare the text data to ensure it is in the optimal format for analysis.
- Chunking:** We split the conversations into manageable chunks by calculating the number of tokens in each conversation. We then group these conversations into chunks, ensuring that the total number of tokens in each chunk does not exceed 1024. This allows the BART model to process the data effectively without exceeding its token limit.

This method ensures that the summarization process is both efficient and accurate, preserving the integrity of the original conversations while fitting within the model's constraints.



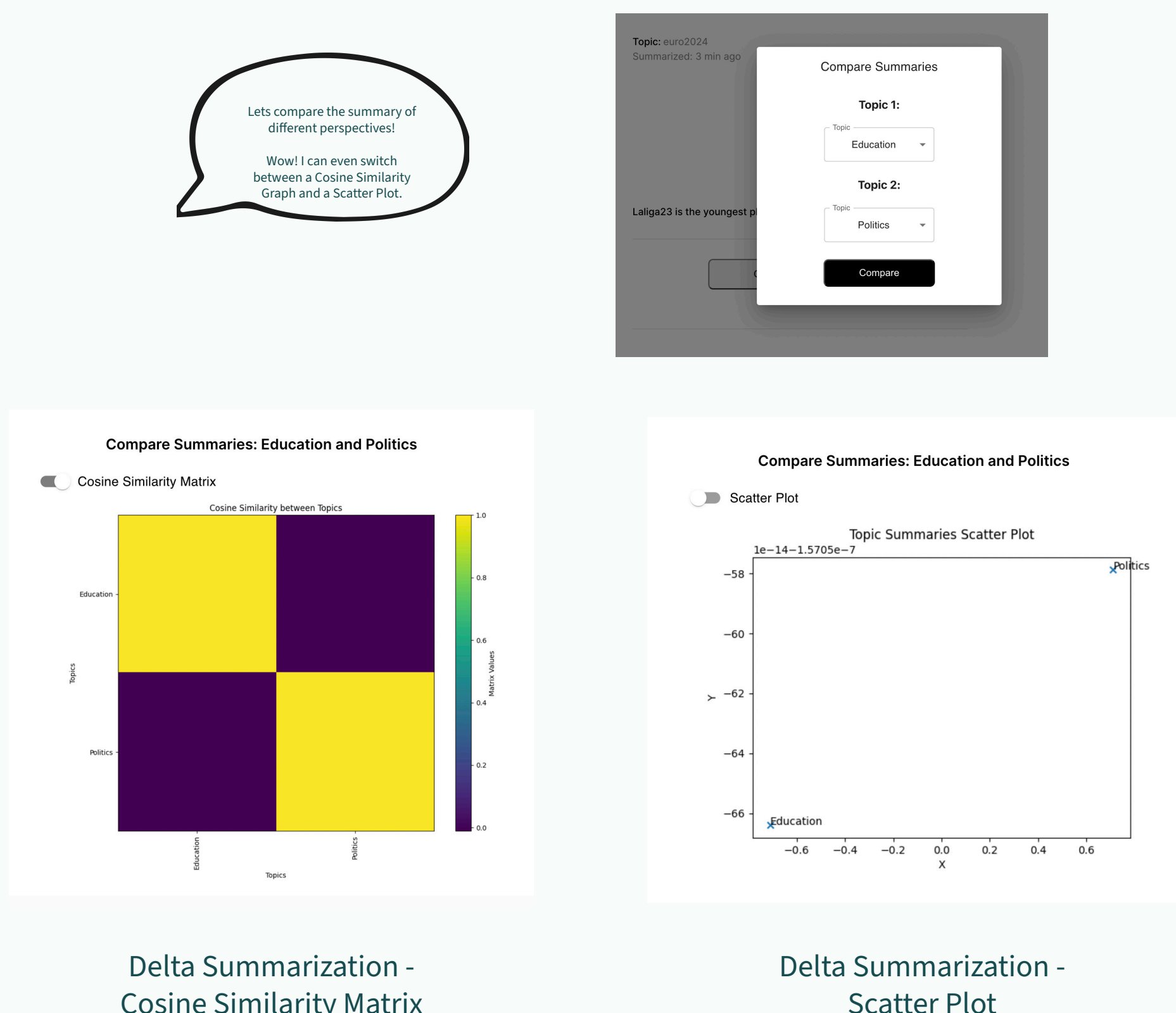
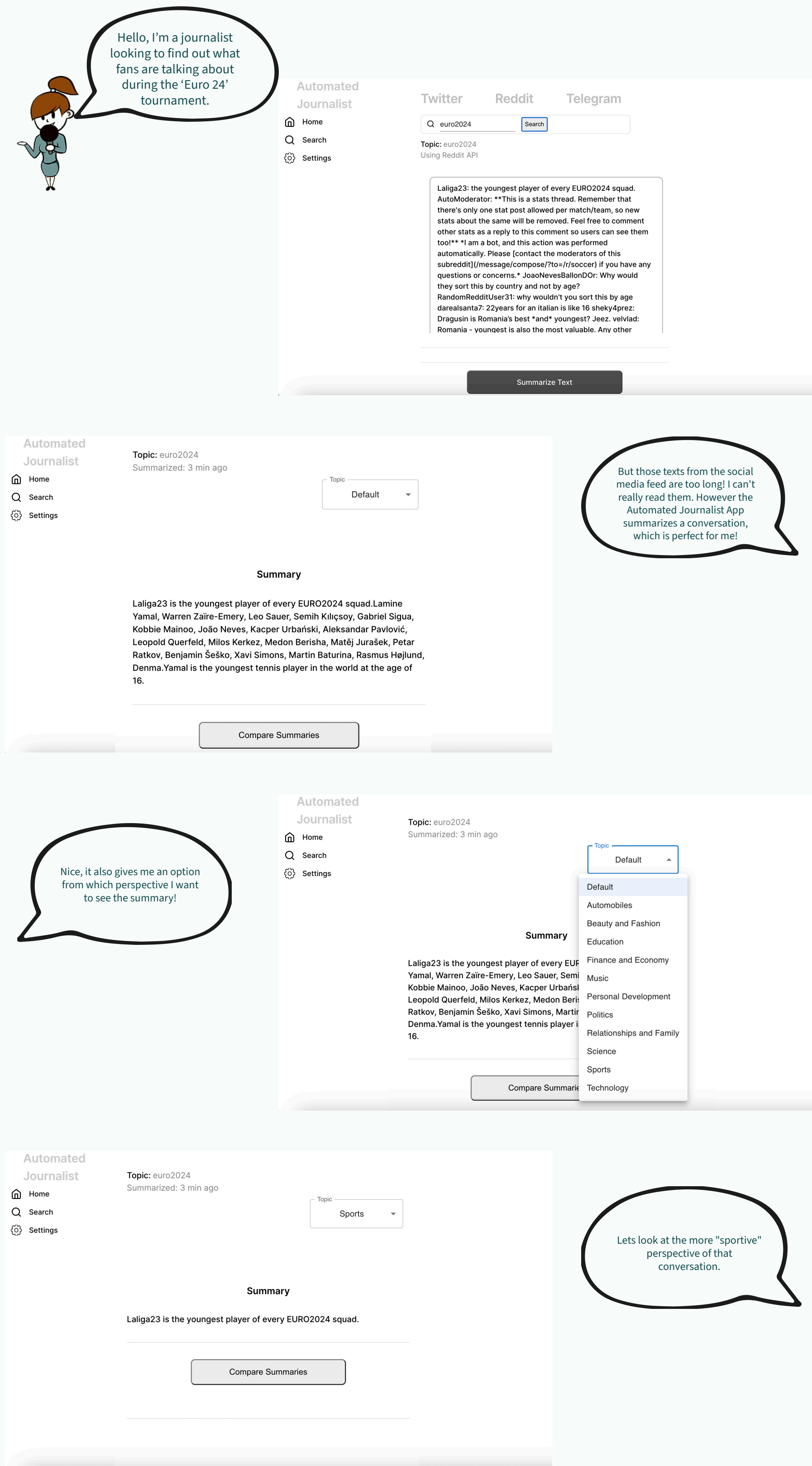
DELTA SUMMARIZATION

To enhance the user experience in comparing different summaries, we avoid displaying complex numerical data directly. Instead, we visualize this information using graphs. We offer two types of graphs:

- Cosine Similarity Graph:** This graph illustrates the cosine similarity between the embeddings of two summaries. It helps users understand how closely related the summaries are in terms of content and context.
- Scatter Plot:** This graph employs Principal Component Analysis (PCA) to reduce the dimensionality of the summary embeddings to two dimensions. It provides a visual representation of the summaries in a simplified space, making it easier to compare and contrast them.

You can also see the specific graphs at the very bottom of the "User Scenario".

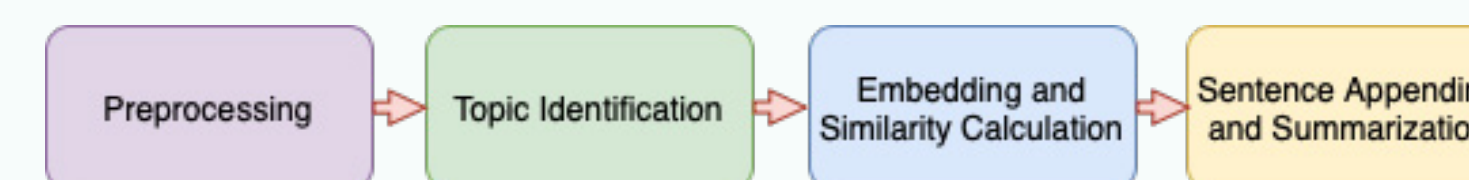
USER SCENARIO



TOPIC AWARE SUMMARIZATION

For topic-aware summarization, we follow a structured approach to ensure accurate and relevant summaries:

- Preprocessing:** The input conversations undergo preprocessing to clean and prepare the text for analysis. This step ensures the data is in the best format for embedding and subsequent processing.
- Topic Identification:** We first identify probable topics using BERTopic. If fewer than three topics are detected, we use predefined static topics. We then generate embeddings for these topics.
- Embedding and Similarity Calculation:** We obtain embeddings for the preprocessed conversations and calculate the cosine similarity between each conversation and the identified topics. This step helps us determine which parts of the conversation are most relevant to each topic.
- Sentence Appending and Summarization:** We append sentences relevant to each topic based on their cosine similarity scores. These topic-specific sentences are then summarized using the BART model to produce abstractive summaries.

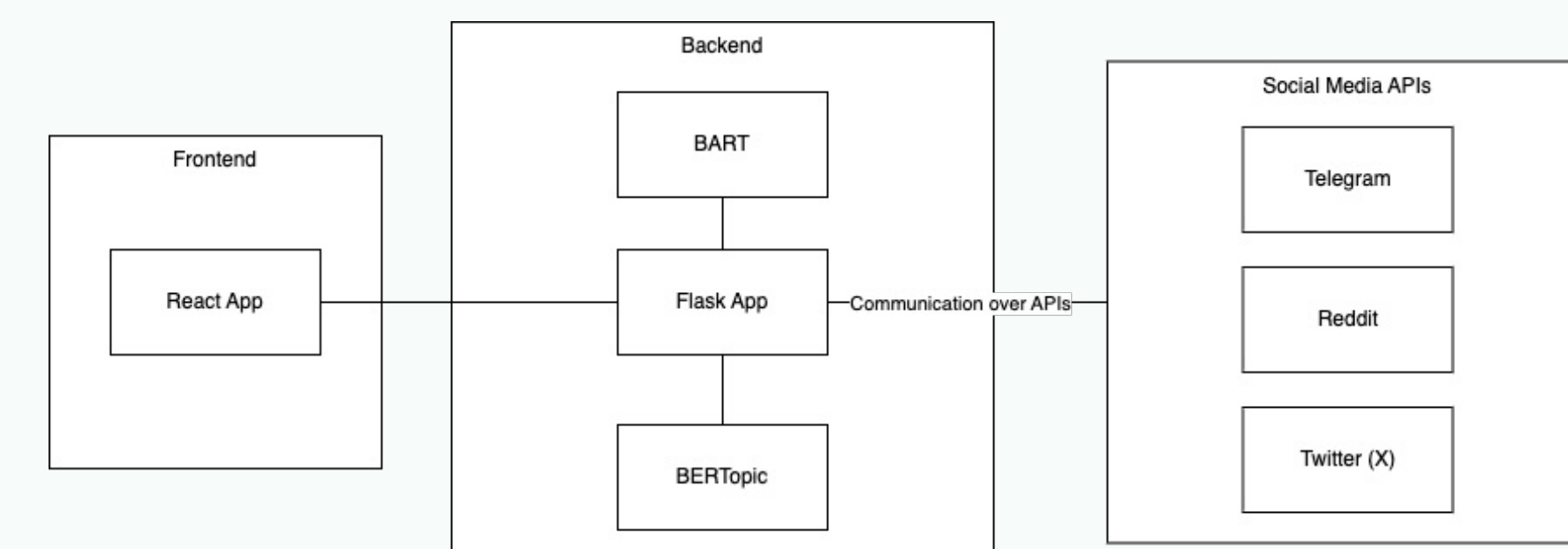


TECHNOLOGY

Our Automated Journalist App consists of two parts: the backend and the frontend. While implementing the app, we also considered the user experience perspective. First, we designed the app using Figma, then we implemented the UI with ReactJS.

For the backend, we extended the code from a previous work using Python Flask. For the summarization part, we used Python with its extensive libraries such as BERTopic and Sentence-Transformers. To fetch social media feeds, we utilized Python libraries such as "Twar" (for X, previously Twitter), "Telethon" (for Telegram), and "Praw" (for Reddit).

To maintain data consistency from different sources, the fetched data is transformed to the Samsun dataset format, which was introduced by Hugging Face. This standardization allows us to use the same summarization techniques on data from different sources directly.



FRONT-END



BACK-END



DESIGN



LET'S LOOK AT THE APP TOGETHER !



We would like to give you the chance to test the demo of our Automated Journalist Application.